

CS30003 – Computer Networks: End Semester Answer Bank

SECTION A: OSI Layer Architecture & Physical Layer

Q1. Compare & Contrast Circuit Switched & Packet Switched Network

Feature	Circuit Switched	Packet Switched
Connection	Dedicated path established before data transfer	No dedicated path; packets routed independently
Resource Allocation	Resources reserved for entire session	Resources allocated on demand
Efficiency	Inefficient (idle resources are wasted)	Efficient (resources shared among users)
Delay	Fixed propagation delay	Variable delay (queuing/processing delay)
Example	Traditional PSTN telephone	Internet (IP networks)
Error Handling	Network ensures error-free transmission	End systems handle errors
Data Unit	Continuous bitstream	Packets (with headers)
Setup Time	Required (call setup phase)	Not required
Cost	Expensive (dedicated resources)	Cheaper (shared infrastructure)

Q2. Line Coding for Binary Stream: 1011101011

Data: 1011101011

(i) Polar RZ (Return to Zero)

- '1' → +V for first half, returns to 0 for second half
- '0' → -V for first half, returns to 0 for second half

$+V$    $-V$  
 0     
 $-V$ (no $-V$ in Polar RZ for '0' in standard form; some variants use $-V$ for 0)

Pattern: $1 \rightarrow +V/0$, $0 \rightarrow -V/0$, return to zero mid-bit.

(ii) AMI (Alternate Mark Inversion)

- '0' $\rightarrow$ 0V (zero level)
- '1' $\rightarrow$ alternates between $+V$ and $-V$

Bit: 1 0 1 1 1 0 1 0 1 1
 AMI: $+V$ 0 $-V$ $+V$ $-V$ 0 $+V$ 0 $-V$ $+V$

(iii) Manchester Encoding

- '1' $\rightarrow$ High-to-Low transition at mid-bit
- '0' $\rightarrow$ Low-to-High transition at mid-bit

Bit: 1 0 1 1 1 0 1 0 1 1


(iv) Differential Manchester

- Transition at beginning of bit period = '0'
- No transition at beginning of bit period = '1'
- Always a transition at the midpoint

(v) Unipolar NRZ

- '1' $\rightarrow +V$
- '0' $\rightarrow$ 0V

Bit: 1 0 1 1 1 0 1 0 1 1
 $+V$ 0 $+V$ $+V$ $+V$ 0 $+V$ 0 $+V$ $+V$

(vi) Polar NRZ (NRZ-L)

- '1' $\rightarrow +V$
- '0' $\rightarrow -V$

Bit: 1 0 1 1 1 0 1 0 1 1
+V -V +V +V +V -V +V -V +V +V

Q3. Transmission Impairments & SNR Calculation

Transmission Impairments

Three main impairments:

1. **Attenuation:** Signal loses strength over distance. Amplifiers/repeaters compensate.
2. **Distortion:** Different frequency components travel at different speeds, causing signal shape change. Common in wideband signals.
3. **Noise:** Unwanted signals added to data signal.
 - *Thermal noise* - due to electron movement
 - *Intermodulation noise* - signals at different frequencies interfere
 - *Crosstalk* - unwanted coupling between adjacent wires
 - *Impulse noise* - sudden spikes (e.g., lightning)

SNR Calculation

- Signal Power (P_s) = 10 mW = 10×10^{-3} W
- Noise Power (P_n) = 1 μ W = 1×10^{-6} W

$$\text{SNR} = P_s / P_n = (10 \times 10^{-3}) / (1 \times 10^{-6}) = 10,000$$

$$\text{SNR (dB)} = 10 \times \log_{10}(10000) = 10 \times 4 = 40 \text{ dB}$$

Q4. Guided & Unguided Media

Guided Media (Wired)

Physical conductor that guides the signal.

1. **Twisted Pair Cable**
 - Two insulated copper wires twisted together
 - UTP (Unshielded) - used in LANs (Cat5, Cat6)
 - STP (Shielded) - reduces interference
 - Speed: up to 10 Gbps (Cat7)
2. **Coaxial Cable**

- Central copper conductor + insulation + metallic shield + outer jacket
- Used in cable TV, broadband
- Better noise immunity than twisted pair

3. Fiber Optic Cable

- Data as light pulses through glass/plastic core
- Single-mode: one ray of light, long distances
- Multi-mode: multiple rays, shorter distances
- Extremely high bandwidth, immune to EMI

Unguided Media (Wireless)

No physical conductor; signal travels through air.

1. **Radio Waves** – 3 kHz to 1 GHz; omnidirectional; used in AM/FM radio, WiFi
2. **Microwaves** – 1 GHz to 300 GHz; directional; used in satellite and terrestrial links
3. **Infrared** – short range; used in TV remotes, IrDA

Q5. Physical Topologies & Mesh Links Calculation

Four Physical Topologies

1. **Bus** – All nodes share a single backbone cable
2. **Star** – All nodes connect to a central hub/switch
3. **Ring** – Each node connected to exactly two others forming a ring
4. **Mesh** – Every node connected to every other node

Mesh Topology Link Calculation

For N nodes in a full mesh:

- **Number of links** = $N(N-1)/2$
- For N = 7: Links = $7 \times 6 / 2 = 21$ links
- Each node requires $N-1 = 6$ ports/interfaces

Q6. OSI Layer Architecture

The OSI (Open Systems Interconnection) model has 7 layers:

Layer	Name	Key Functions
7	Application	User interface, HTTP, FTP, SMTP, DNS
6	Presentation	Data translation, encryption, compression
5	Session	Session management, synchronization
4	Transport	End-to-end delivery, segmentation, TCP/UDP
3	Network	Logical addressing, routing, IP
2	Data Link	Framing, MAC addressing, error detection
1	Physical	Bit transmission, signal encoding, cable specs

Data units: Application→Data, Transport→Segment, Network→Packet, Data Link→Frame, Physical→Bits.

Q7. TCP/IP Protocol Stack

Layer	Protocols	Address Used
Application	HTTP, FTP, SMTP, DNS, DHCP	Domain Name / URL
Transport	TCP, UDP	Port Number (16-bit)
Internet	IP, ICMP, ARP, OSPF, BGP	IP Address (32-bit)
Network Access	Ethernet, Wi-Fi, PPP	MAC Address (48-bit)

TCP/IP collapses OSI layers 5,6,7 into Application; layers 1,2 into Network Access.

Q8. Parallel & Serial Transmission

Parallel Transmission

- Multiple bits transmitted simultaneously over multiple wires
- n wires → n bits per clock cycle
- Faster but expensive over long distances; susceptible to skew

Asynchronous Serial Transmission

- Data sent one bit at a time
- No shared clock between sender and receiver

- Each character framed by **Start bit (0)** and **Stop bit (1)**
- Gaps allowed between characters
- Example: RS-232 serial communication

Synchronous Serial Transmission

- Continuous stream of bits with shared clock signal
 - No start/stop bits per character; uses sync characters at frame start
 - More efficient for bulk data transfer
 - Example: HDLC, SONET
-

Q9. Synchronous TDM - 4 lines of 100 kbps, 10 bits multiplexed

Given: 4 input lines, each 100 kbps; 10 bits per slot

i. Input Time Slot

= bits/line per slot ÷ line rate = 10 bits / 100 kbps = **0.1 ms**

ii. Output Channel Capacity (Data Rate)

= Number of lines × Input data rate = 4 × 100 kbps = **400 kbps**

iii. Output Time Slot

= 10 bits / 400 kbps = **0.025 ms**

iv. Frame Duration

= Number of slots × Output time slot = 4 × 0.025 ms = **0.1 ms**

v. Frame Rate

= 1 / Frame Duration = 1 / 0.1 ms = **10,000 frames/second**

Q10. Statistical TDM - 10 lines of 9600 bps, 60% busy, utilization 0.9

Given: 10 lines × 9600 bps; busy factor (ρ) = 0.60; desired utilization = 0.9

i. Channel Capacity for Statistical TDM

- Total offered load = 10 × 9600 × 0.60 = **57,600 bps**
- For utilization ≤ 0.9 : Required capacity = 57,600 / 0.9 = **64,000 bps = 64 kbps**

ii. Load on Multiplexer

$$= 10 \times 9600 \times 0.60 = \mathbf{57,600 \text{ bps}}$$

iii. Saving vs. Synchronous TDM

- Synchronous TDM capacity = $10 \times 9600 = \mathbf{96,000 \text{ bps}}$
 - Statistical TDM capacity = $\mathbf{64,000 \text{ bps}}$
 - Saving = $96,000 - 64,000 = \mathbf{32,000 \text{ bps}}$ (33.3% saving)
-

SECTION B: Data Link Layer (Layer II)

Q1. Hamming Code (7-bit) for Data = 1011, Even Parity

Data bits (d): d4=1, d3=0, d2=1, d1=1

Parity bit positions: p1 (bit 1), p2 (bit 2), p4 (bit 4)

Positions:

Bit pos:	7	6	5	4	3	2	1
	d4	d3	d2	p4	d1	p2	p1
	1	0	1	p4	1	p2	p1

Calculate p1 (covers positions 1,3,5,7): bits at 1,3,5,7 $\rightarrow$ p1,1,1,1

- For even parity: $p1 + 1 + 1 + 1 = \text{even} \rightarrow p1 = 1$

Calculate p2 (covers positions 2,3,6,7): p2,1,0,1

- $p2 + 1 + 0 + 1 = \text{even} \rightarrow p2 = 0$

Calculate p4 (covers positions 4,5,6,7): p4,1,0,1

- $p4 + 1 + 0 + 1 = \text{even} \rightarrow p4 = 0$

7-bit Hamming Code: 1 0 1 0 0 1 1 (positions 7 to 1)

Error Detection (3rd bit from right corrupted)

Received codeword: bit 3 flipped $\rightarrow$ 1 0 1 0 1 1 1

Syndrome bits:

- s1 (check p1, pos 1,3,5,7): $1+1+1+1 = 4 \rightarrow \text{even} \rightarrow s1=0$

- s_2 (check p_2 , pos 2,3,6,7): $1+1+0+1 = 3 \rightarrow \text{odd} \rightarrow s_2=1$
- s_4 (check p_4 , pos 4,5,6,7): $0+1+0+1 = 2 \rightarrow \text{even} \rightarrow s_4=0$

Syndrome = $s_4 s_2 s_1 = 010$ (binary) = 2 (decimal)

→ **Error in bit position 2** — receiver corrects it.

(Note: "3rd bit from right" = bit position 3, so syndrome should point to position 3 = $s_4 s_2 s_1 = 011 = 3$)

Q2. CRC: Data = $x^5+x^3+x^2+1$, Divisor = x^4+x+1

Data polynomial: $x^5+x^3+x^2+1 \rightarrow$ binary: **101101** **Divisor:** $x^4+x+1 \rightarrow$ binary: **10011** **Degree of divisor (r): 4**

Augmented data (append $r=4$ zeros): 1011010000

Binary Division (XOR):

```
1011010000 ÷ 10011
  10011
  -----
    10000
    10011
    -----
      00111 00
      (continue XOR division...)
```

After complete division, **Remainder (FCS) = 1110**

Codeword = Data + FCS = 1011011110

At Receiver (no error): Divide codeword by divisor → Syndrome = **0000** (no error)

Q3. CRC: Data = 100101, Divisor = 1101

Augmented data (append 3 zeros as divisor is 4 bits): 100101000

XOR Division:


```

100101000 ÷ 1101
1101
----
 1001
 1101
 ----
   1000
   1101
   ----
    1011
    1101
    ----
     1100
     1101
     ----
      001  ← Remainder

```

Remainder = 001 (this is appended as FCS at sender)

Q4. 16-bit Checksum for "Forouzan"

ASCII (HEX) values: F=46, o=6F, r=72, o=6F, u=75, z=7A, a=61, n=6E

Pair into 16-bit words:

- Fo = 0x466F
- ro = 0x726F
- uz = 0x757A
- an = 0x616E

Sum:

```

    0x466F
+ 0x726F
-----
    0xB8DE

+ 0x757A
-----
0x12E58 → carry: 0x2E58 + 0x0001 = 0x2E59

+ 0x616E
-----
    0x8FC7

```

Checksum = 1's complement of 0x8FC7 = 0x7038

Q5. Go-Back-N vs. Selective Repeat ARQ

Feature	Go-Back-N (GBN)	Selective Repeat (SR)
Window Size	Sender window = $2^m - 1$	Sender & Receiver window = 2^m
Retransmission	All frames from lost frame retransmitted	Only the lost/errored frame retransmitted
Receiver Buffer	No buffering of out-of-order frames	Out-of-order frames buffered
Efficiency	Lower (wastes bandwidth)	Higher (no unnecessary retransmission)
Complexity	Simpler	More complex
ACK Type	Cumulative ACK	Selective/Individual ACK

Q6. Go-Back-N ARQ with m=3

m=3 → Window size = $2^3 - 1 = 7$

Scenario:

- Frames 0,1,2 → received successfully; ACKs 1,2,3 sent
- Frame 3 → LOST (noise)
- Frames 4,5 → transmitted (within window); ACKs 4,5 received but ACK 6 (for frame 3) never came
- Timeout → Sender goes back to Frame 3 and retransmits 3,4,5,6,...

```
Sender:  [0][1][2][3][4][5][X:ACK6 lost]→retransmit[3][4][5]...
```

↑LOST

```
Receiver: ACK1 ACK2 ACK3      ACK5 ACK6 → discards 4,5 after timeout
```

Sf (Send window front) advances with each new ACK. On loss of Frame 3 and missing ACK, Sf stays at 3, Sn (next to send) resets to 3.

Q7 & Q8. Selective Repeat ARQ with m=3

Window size = $2^{(m-1)} = 4$

Scenario:

- Frame 0,1 → received; ACK 1,2 sent
- Frame 2 → LOST
- Frame 3 → received and **buffered** (out of order)
- ACK 5 for Frame 3 not received at sender

Key difference from GBN: Only Frame 2 is retransmitted. Frame 3 remains in receiver buffer and is delivered once Frame 2 arrives. No unnecessary retransmission.

Q9. Media Access Control (MAC)

MAC is a sublayer of the Data Link Layer that controls how devices in a shared medium access the channel.

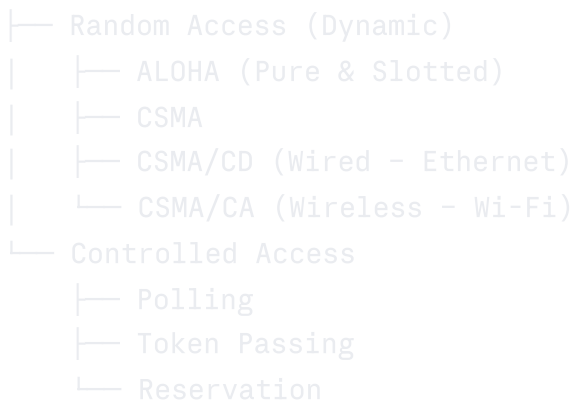
Why required?

- When multiple nodes share the same transmission medium (e.g., Ethernet, Wi-Fi), collisions occur if two send simultaneously.
 - MAC protocols coordinate channel access to prevent or handle collisions, ensuring fair and efficient use of the shared channel.
-

Q10. Classification of MAC Protocols

MAC Protocols

```
├─ Channelization (Static)
|   ├─ FDMA (Frequency Division)
|   ├─ TDMA (Time Division)
|   └─ CDMA (Code Division)
```



Q11. Pure & Slotted ALOHA

Pure ALOHA

- Any station transmits whenever it has data
- If collision → wait random time → retransmit
- **Vulnerable time = $2T$** (T = frame transmission time)
- **Throughput $S = G \times e^{(-2G)}$; Maximum $S = 18.4\%$ at $G=0.5$**

Slotted ALOHA

- Time divided into slots equal to frame transmission time
- Station can only transmit at the beginning of a slot
- Reduces collision window
- **Vulnerable time = T**
- **Throughput $S = G \times e^{(-G)}$; Maximum $S = 36.8\%$ at $G=1$**

Q12. Pure ALOHA Throughput for 200-bit frames on 200 kbps

Frame transmission time $T = 200 \text{ bits} / 200,000 \text{ bps} = 1 \text{ ms}$

$S = G \times e^{(-2G)}$ where $G = \lambda T$ (offered load)

a. 1000 frames/sec

$G = 1000 \times 0.001 = 1$ $S = 1 \times e^{(-2)} = 0.135 \rightarrow 13.5\%$ Throughput = $0.135 \times 200 \text{ kbps} = 27 \text{ kbps}$

b. 500 frames/sec

$G = 500 \times 0.001 = 0.5$ $S = 0.5 \times e^{(-1)} = 0.5 \times 0.368 = 0.184 \rightarrow 18.4\%$ (maximum) Throughput = **36.8 kbps**

c. 250 frames/sec

$$G = 250 \times 0.001 = 0.25 \text{ S} = 0.25 \times e^{(-0.5)} = 0.25 \times 0.607 = 0.152 \rightarrow \mathbf{15.2\% \text{ Throughput} = 30.4 \text{ kbps}}$$

Q13. CSMA/CA (Wireless)

CSMA/CA = Carrier Sense Multiple Access with Collision Avoidance

Used in IEEE 802.11 (Wi-Fi) because collision *detection* is impossible in wireless (hidden terminal problem, signal fading).

Three Strategies:

1. **IFS (Interframe Space):** Stations wait an IFS period before transmitting to give priority (SIFS < PIFS < DIFS)
2. **Contention Window (Backoff):** If channel busy, station picks random backoff in range [0, CW-1]; countdown while channel is idle
3. **ACKnowledgement:** Receiver sends ACK for every frame; if no ACK received, retransmit

Flowchart (simplified):

```
Data ready → Sense channel
├─ Idle for DIFS → Transmit
├─ Busy → Wait until idle + DIFS → Start backoff timer
│   └─ Timer reaches 0 → Transmit
│       └─ Channel goes busy → Freeze timer → Resume when idle
```

Q14. CSMA/CD (Wired Ethernet)

CSMA/CD = Carrier Sense Multiple Access with Collision Detection

Working:

1. Sense channel before transmitting
2. If idle → transmit
3. While transmitting, continuously monitor for collision
4. Collision detected → send jam signal → stop transmission
5. Wait for random backoff time (Binary Exponential Backoff)
6. Retransmit

Why collision detection is NOT possible in wireless:

- In wireless, a transmitting station cannot hear other signals while transmitting (its own strong signal drowns out weak incoming signals)

- Hidden terminal problem: two stations can't hear each other but both can collide at the AP
 - Signal strength degrades rapidly with distance
-

Q15. Classification of Ethernets

Standard	Speed	Medium
10Base-T	10 Mbps	UTP (Twisted Pair)
10Base-2 (Thinnet)	10 Mbps	Coaxial
10Base-5 (Thicknet)	10 Mbps	Coaxial
100Base-T (Fast Ethernet)	100 Mbps	UTP
100Base-FX	100 Mbps	Fiber
1000Base-T (Gigabit)	1 Gbps	UTP
1000Base-F	1 Gbps	Fiber
10GBase-T	10 Gbps	UTP (Cat6a/Cat7)
10GBase-SR/LR	10 Gbps	Fiber

Q16. Cable Standards

100Base-T (Fast Ethernet)

- Speed: 100 Mbps
- Medium: Category 5 UTP (2 pairs)
- Max Segment: 100 meters
- Encoding: 4B/5B with MLT-3
- Connector: RJ-45

1000Base-F (Gigabit Ethernet over Fiber)

- Speed: 1000 Mbps (1 Gbps)
- Medium: Fiber optic (Single-mode or Multi-mode)
- 1000Base-SX: Multi-mode, up to 550 m
- 1000Base-LX: Single-mode, up to 5 km

- Encoding: 8B/10B
 - Connector: SC or LC
-

SECTION C: Network Layer

Q1. Network Layer Responsibilities

Responsibilities:

- **Logical Addressing:** Assigns IP addresses to uniquely identify hosts across networks
 - **Routing:** Determines best path for packet from source to destination using routing algorithms (RIP, OSPF, BGP)
 - **Packet Forwarding:** Moving packet from input interface to correct output interface
 - **Fragmentation & Reassembly:** Breaking large packets for smaller MTU networks
 - **Error Reporting:** Via ICMP
-

Q2. Subnetting: 172.18.0.0 into 7 subnets of 8190 hosts each

Given: Network: 172.18.0.0 (Class B), Need: 7 subnets, 8190 hosts each

Host bits needed: $2^n - 2 \geq 8190 \rightarrow n=13$ ($2^{13}-2 = 8190$ ✓)

Subnet bits: $32 - 16$ (class B) $- 13$ (host) = **3 subnet bits** $\rightarrow$ supports $2^3 = 8$ subnets

Subnet mask: /19 (255.255.224.0) **Block size:** $2^{13} = 8192$ addresses per subnet

Location	Network Address	Broadcast	Usable IP Range
Mumbai	172.18.0.0/19	172.18.31.255	172.18.0.1 - 172.18.31.254
Delhi	172.18.32.0/19	172.18.63.255	172.18.32.1 - 172.18.63.254
Chennai	172.18.64.0/19	172.18.95.255	172.18.64.1 - 172.18.95.254
Bangalore	172.18.96.0/19	172.18.127.255	172.18.96.1 - 172.18.127.254
Hyderabad	172.18.128.0/19	172.18.159.255	172.18.128.1 - 172.18.159.254
Pune	172.18.160.0/19	172.18.191.255	172.18.160.1 - 172.18.191.254
Bhubaneswar	172.18.192.0/19	172.18.223.255	172.18.192.1 - 172.18.223.254

Q3. VLSM: 192.168.10.0/24 into 4 subnets

Requirements: S1=126, S2=62, S3=30, S4=30 hosts

Strategy: Allocate largest subnet first.

Subnet 1: 126 hosts

- Need 7 host bits ($2^7-2=126$), prefix = /25
- **Network:** 192.168.10.0/25
- **Broadcast:** 192.168.10.127
- **Range:** 192.168.10.1 - 192.168.10.126

Subnet 2: 62 hosts

- Need 6 host bits ($2^6-2=62$), prefix = /26
- **Network:** 192.168.10.128/26
- **Broadcast:** 192.168.10.191
- **Range:** 192.168.10.129 - 192.168.10.190

Subnet 3: 30 hosts

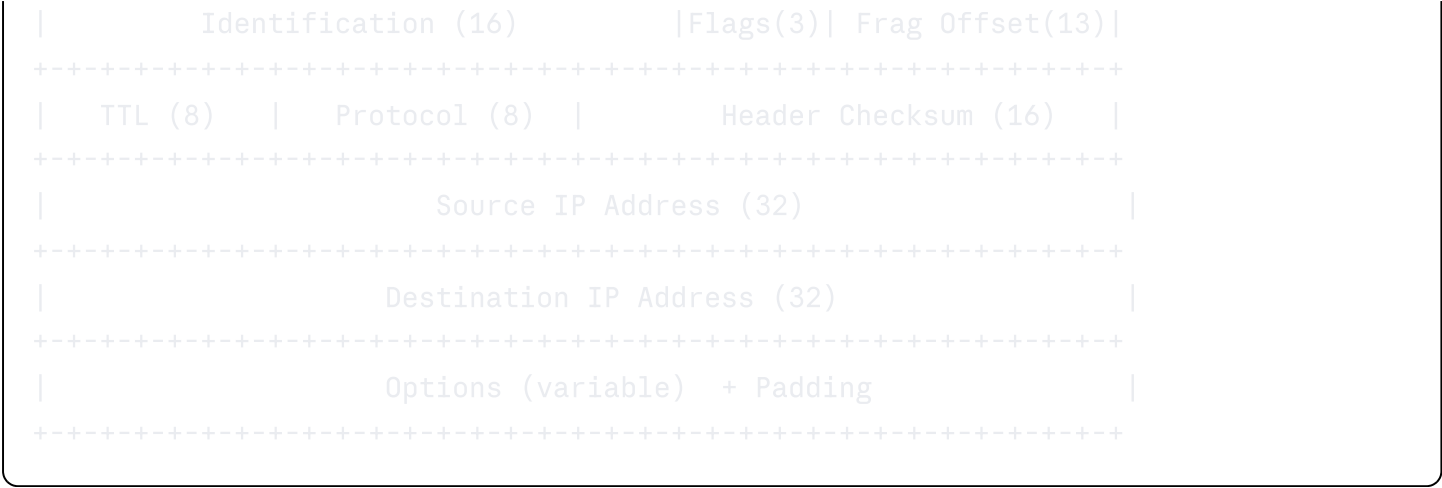
- Need 5 host bits ($2^5-2=30$), prefix = /27
- **Network:** 192.168.10.192/27
- **Broadcast:** 192.168.10.223
- **Range:** 192.168.10.193 - 192.168.10.222

Subnet 4: 30 hosts

- **Network:** 192.168.10.224/27
 - **Broadcast:** 192.168.10.255
 - **Range:** 192.168.10.225 - 192.168.10.254
-

Q4. IPv4 Header Structure





Field	Size	Description
Version	4 bits	IP version (4 for IPv4)
IHL	4 bits	Header length in 32-bit words (min=5)
DSCP/TOS	8 bits	Differentiated services/QoS
Total Length	16 bits	Total packet size (header + data), max 65535
Identification	16 bits	Fragment group identifier
Flags	3 bits	DF (Don't Fragment), MF (More Fragments)
Fragment Offset	13 bits	Position of fragment in original datagram (in 8-byte units)
TTL	8 bits	Hop count; decremented at each router
Protocol	8 bits	Upper layer: 6=TCP, 17=UDP, 1=ICMP
Header Checksum	16 bits	Error check on header only
Source IP	32 bits	Sender IP address
Destination IP	32 bits	Receiver IP address

Q5. IPv4 Packet Parsing: 0x480200A110302150FC001000

Hex: 48 02 00 A1 10 30 21 50 FC 00 10 00

Field	Hex	Value
Version	4	IPv4
IHL	8	$8 \times 4 = 32$ bytes header
DSCP/TOS	02	DSCP=0, ECN=2
Total Length	00A1	161 bytes
Identification	1030	4144
Flags+Frag Offset	2150	Binary: 0010 0001 0101 0000 → Flags=001 (MF=1, DF=0), Offset = 0001 0101 0000 = $0x150 = 336 \rightarrow 336 \times 8 = 2688$ bytes
TTL	FC	252
Protocol	00	HOPOPT (or reserved)
Checksum	1000	0x1000

Q6. IPv4 Fragmentation: 4200 bytes, ID=15460

Total data = 4200 bytes; First two fragments = 1600 bytes each; Third = 1000 bytes Header = 20 bytes standard

Fragment	Data	Total Length	ID	MF	Offset
FRAG 1	1600	1620	15460	1	0
FRAG 2	1600	1620	15460	1	200 (1600/8)
FRAG 3	1000	1020	15460	0	400 (3200/8)

FRAG 2 further fragmented into equal halves (800 bytes each):

Sub-Fragment	Data	Total Length	ID	MF	Offset
FRAG 2.1	800	820	15460	1	200
FRAG 2.2	800	820	15460	1	300 (2400/8)

Q7. Fragmentation: 5000 bytes, ID=15671

Total packet = 5000 bytes; Header = 20 bytes; Data = 4980 bytes

Fragment 1: MTU=1420 bytes → Data = 1420-20 = 1400 bytes

Frag	Data	Total	ID	MF	Offset
F1	1400	1420	15671	1	0

Fragment 2: MTU=1620 → Data = 1600 bytes, but then further split into two with 1000 bytes each

Sub-Frag	Data	Total	ID	MF	Offset
F2.1	1000	1020	15671	1	175 (1400/8)
F2.2	600	620	15671	1	300 ((1400+1000)/8)

Fragment 3: MTU=1020 → Data = 1000 bytes; Offset = (1400+1600)/8 = 375

Fragment 4: Remaining data = 4980-1400-1600-1000 = 980 bytes; Offset = (4000/8) = 500; MF=0

Q8. Fragmentation: 6000-byte packet, Header=30 bytes, MTU=1400

Data = 6000 - 30 = 5970 bytes Max data per fragment = 1400 - 20 = 1380 bytes (standard 20-byte header used in fragmentation)

(Note: though header is 30 bytes originally, fragments use same 20-byte minimum)

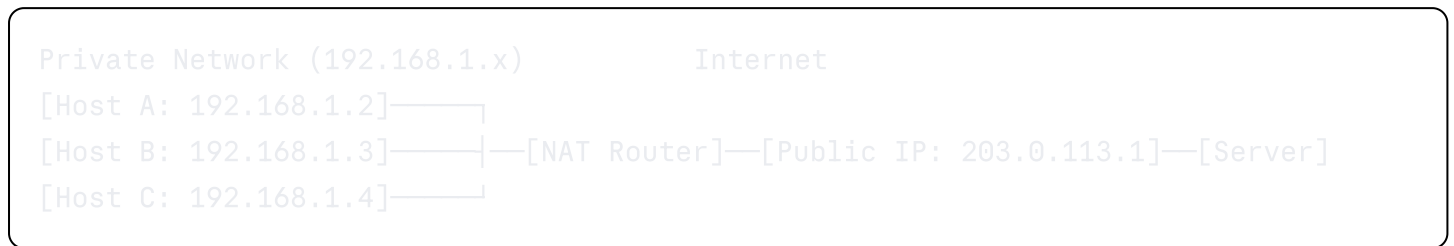
Fragment	Data Bytes	Total Length	MF Flag	Offset (÷8)
F1	1380	1400	1	0
F2	1380	1400	1	172 (1376/8... adjust to multiple of 8: 1376)
F3	1380	1400	1	344
F4	1380	1400	1	516
F5	450	470	0	688

(Exact calculation: $5970/1380 = 4 \text{ full} + 1 \text{ partial}$)

Q9. NAT (Network Address Translation)

Requirement: IPv4 addresses are limited (~4.3 billion). NAT allows multiple private hosts to share one public IP address.

Diagram:



NAT maintains a **NAT translation table** mapping (Private IP, Port) ↔ (Public IP, Port).

Types:

- **Static NAT:** 1-to-1 mapping
 - **Dynamic NAT:** Pool of public IPs
 - **PAT/NAPT:** Many-to-one (most common), uses port numbers to distinguish
-

Q10. ARP (Address Resolution Protocol)

Requirement: Devices know destination IP but need MAC address to build Layer 2 frame. ARP maps IP → MAC.

Working:

1. Host A wants to send to IP 192.168.1.5 (unknown MAC)
2. A broadcasts **ARP Request**: "Who has 192.168.1.5? Tell 192.168.1.2"
3. Host with 192.168.1.5 replies with **ARP Reply** (unicast): "I have 192.168.1.5, my MAC is XX:XX:XX:XX:XX:XX"
4. A caches the mapping in its ARP cache

Gratuitous ARP: Host announces its own IP→MAC mapping.

Q11. ICMP Query Messages

ICMP (Internet Control Message Protocol) is used for error reporting and network diagnostics.

Query Type	Code	Description
Echo Request (Ping)	Type 8	Tests reachability; receiver must reply with Echo Reply
Echo Reply	Type 0	Response to Echo Request
Timestamp Request	Type 13	Measures round-trip time
Timestamp Reply	Type 14	Response to timestamp request
Address Mask Request	Type 17	Host asks router for subnet mask
Address Mask Reply	Type 18	Router responds with mask
Router Solicitation	Type 10	Host asks for router info
Router Advertisement	Type 9	Router announces itself

Q12. NAT Working Example

NAT Router R1 Public IP: 10.63.10.113 **Private Network:** 192.168.1.0/24 (Gateway: 192.168.1.1)
Host: 192.168.1.37 **Server:** 47.32.172.11 (www.kiit.ac.in)

Step 1 (Outgoing):

- Host sends packet: Src=192.168.1.37:3456, Dst=47.32.172.11:80
- NAT translates: Src → 10.63.10.113:5001, Dst unchanged
- NAT table entry: (192.168.1.37:3456 ↔ 10.63.10.113:5001)

Step 2 (Incoming Reply):

- Server replies: Src=47.32.172.11:80, Dst=10.63.10.113:5001
- NAT translates Dst back: Dst → 192.168.1.37:3456
- Packet delivered to host

Q13. ARP Packet Format

Field	Size	Description
Hardware Type	16 bits	1 = Ethernet
Protocol Type	16 bits	0x0800 = IPv4

Field	Size	Description
Hardware Address Length	8 bits	6 (MAC = 6 bytes)
Protocol Address Length	8 bits	4 (IPv4 = 4 bytes)
Operation	16 bits	1=Request, 2=Reply
Sender MAC	48 bits	Requester's MAC
Sender IP	32 bits	Requester's IP
Target MAC	48 bits	0 in request, filled in reply
Target IP	32 bits	Desired IP

Encapsulation: ARP packet is placed inside Ethernet frame with EtherType 0x0806. Destination MAC = FF:FF:FF:FF:FF:FF (broadcast) for request.

Q14. ARP vs RARP: Cross-network scenario

Host A: 192.168.1.100/24, MAC E1:34:2A:BC:1F:CA **Host B:** 10.1.23.73/8, Router R1 in between
 Since Host A and B are on **different networks**, ARP cannot directly resolve B's MAC.

Protocol used: ARP is used, but Host A ARPs for **Router R1's MAC** (default gateway), not Host B's.

Process:

1. Host A checks: Is 10.1.23.73 in my subnet (192.168.1.x/24)? → NO
2. Host A sends ARP Request for its **default gateway** (R1's interface on 192.168.1.0/24)
3. R1 replies with its MAC address
4. Host A sends packet to R1 (MAC of R1, IP of Host B)
5. R1 routes the packet; ARPs on its other interface for Host B's MAC
6. Packet delivered to Host B

Q15. DHCP (Dynamic Host Configuration Protocol)

Purpose: Automatically assigns IP address, subnet mask, gateway, DNS to hosts.

DHCP Packet Fields:

Field	Description
op	1=Request, 2=Reply
htype	Hardware type (1=Ethernet)
hlen	Hardware address length
xid	Transaction ID
ciaddr	Client IP address
yiaddr	Your (offered) IP address
siaddr	Server IP
chaddr	Client hardware (MAC) address
options	Includes message type, lease time, etc.

8 DHCP Message Types (Options):

- 1. DHCPDISCOVER - Client broadcasts to find server
- 2. DHCPOFFER - Server offers IP address
- 3. DHCPREQUEST - Client requests offered IP
- 4. DHCPACK - Server confirms assignment
- 5. DHCPNAK - Server declines request
- 6. DHCPDECLINE - Client declines offer (conflict)
- 7. DHCPRELEASE - Client releases IP
- 8. DHCPINFORM - Client requests config info (already has IP)

DHCP Exchange (DORA):



Q16. Distance Vector vs. Link-State Routing

Feature	Distance Vector	Link-State
Algorithm	Bellman-Ford	Dijkstra
Information shared	Routing table with neighbors	Full topology (LSA flooded)
Convergence	Slow; count-to-infinity problem	Fast
Bandwidth usage	Low (periodic updates)	High initially (flooding)
Example	RIP	OSPF, IS-IS
Knowledge	Local (next-hop only)	Global (complete topology)

Q17. Bellman-Ford (Distance Vector) - Shortest Path to N6

Algorithm: For each node v : $D(v) = \min \text{ over all neighbors } u \text{ of } \{ \text{cost}(v,u) + D(u) \}$

Initialization: $D(N6) = 0$, all others $= \infty$

Iterate until no change.

(Exact distances depend on the graph image, which is embedded in the original document. Apply Bellman-Ford iterations: initialize source node to 0, all others to infinity, then relax all edges $N-1$ times.)

Key Steps:

1. Set $D[N6]=0$, $D[\text{all others}]=\infty$
2. For each edge (u,v,w) : If $D[u] + w < D[v]$, update $D[v] = D[u] + w$
3. Repeat for $|V|-1$ iterations

Q18. Dijkstra's Algorithm (Link-State) - Node A to all others

Algorithm:

1. Initialize: $D(A)=0$, all others $=\infty$, $S=\{\}$
2. Pick node u not in S with min $D(u)$, add to S
3. Update: For each neighbor v of u : $D(v) = \min(D(v), D(u)+\text{cost}(u,v))$
4. Repeat until all nodes in S

Table format:

Iteration	S (Confirmed)	D(B),p	D(C),p	D(D),p	D(E),p	D(F),p
Init	{A}	(from A)	...	...	...	...
1	{A, nearest}	updated	updated	...	...	...

(Fill with actual values from the graph in the question.)

SECTION D: Transport Layer

Q1. Transport Layer Features & Responsibilities

- End-to-End Communication:** Provides logical connection between processes (not hosts)
- Multiplexing/Demultiplexing:** Uses port numbers to direct data to correct application
- Segmentation & Reassembly:** Breaks application data into segments, reassembles at destination
- Flow Control:** Prevents sender from overwhelming receiver (TCP sliding window)
- Error Control:** Detects and retransmits lost/corrupted segments (TCP)
- Congestion Control:** Manages network traffic levels (TCP only)
- Connection Management:** TCP establishes/terminates connections (3-way handshake)
- Reliability:** TCP provides guaranteed, ordered delivery; UDP does not

Q2. IANA Port Number Ranges

Range	Type	Usage
0-1023	Well-Known Ports	Standard services: HTTP(80), HTTPS(443), FTP(21), SSH(22), SMTP(25), DNS(53)
1024-49151	Registered Ports	Registered apps: MySQL(3306), RDP(3389), PostgreSQL(5432)
49152-65535	Dynamic/Ephemeral	Temporarily assigned by OS for client-side connections

Q3. UDP Pseudo Header

The pseudo header is used only for **checksum computation** (not transmitted):

```
+-----+
| Source IP Address (32 bits) |
+-----+
| Destination IP Address (32 bits) |
+-----+
| Zeros (8) | Protocol=17 (8 bits) |
+-----+
| UDP Length (16 bits) |
+-----+
```

Why IP address in Transport Layer? The checksum covers the pseudo header (including IP addresses) to verify the packet was delivered to the correct host. It's a cross-layer check to detect misdelivered packets.

Q4. UDP Checksum for "Electronics" (server 10.16.1.17:13 → client 192.168.1.100:1597)

ASCII for "Electronics": E=45, l=6C, e=65, c=63, t=74, r=72, o=6F, n=6E, i=69, c=63, s=73

UDP Header:

- Source Port: 13 = 0x000D
- Dest Port: 1597 = 0x063D
- Length: 8 (header) + 11 (data) = 19 = 0x0013
- Checksum: computed below

Pseudo Header + UDP segment words (16-bit pairs):

```
Source IP: 10.16.1.17 = 0A10 0111
Dest IP: 192.168.1.100 = C0A8 0164
Protocol+zeros: 0x0011
UDP Length: 0x0013
Src Port: 0x000D
Dst Port: 0x063D
Length: 0x0013
Data pairs: 456C 6563 7472 6F6E 6963 7300 (pad to even)
```

Sum all 16-bit words, add carries, take 1's complement.

(Full hex calculation yields the UDP checksum; the method is as above. Key is to pair data bytes, include pseudo header, sum everything with wraparound carry, and complement.)

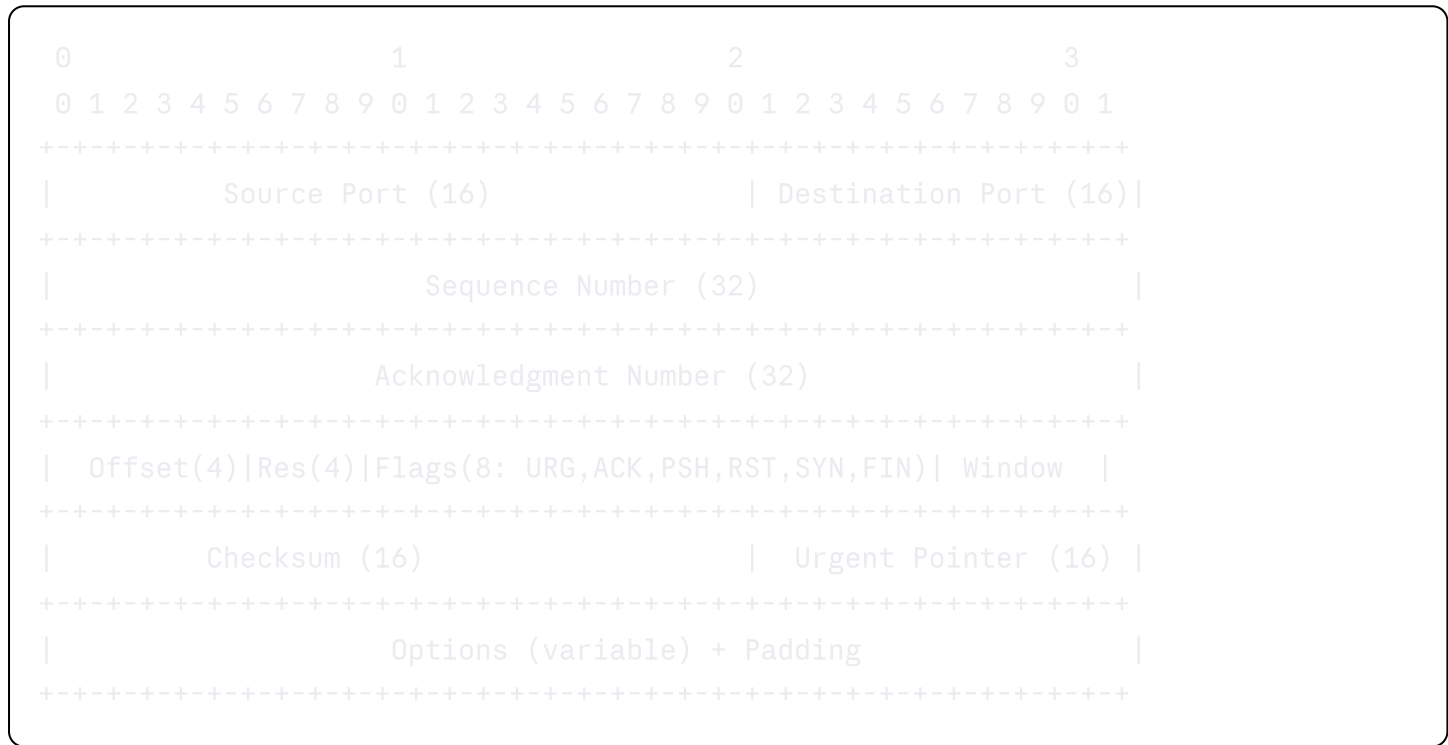
Q5. UDP Checksum for "CoronaVirus"

Same approach as Q4: **ASCII:** C=43, o=6F, r=72, o=6F, n=6E, a=61, V=56, i=69, r=72, u=75, s=73

Source Port: 13 = 0x000D **Dest Port:** 1635 = 0x0663 **Length:** 8 + 11 = 19 = 0x0013

Apply pseudo header checksum computation as described in Q4 with these values.

Q6. TCP Header Format



b. Data Transfer

```
Client — Data (Seq=x+1, Ack=y+1) —> Server
Client <— ACK (Seq=y+1, Ack=x+1+n) — Server
```

c. Connection Termination (4-Way / sometimes 3-Way)

```
Client                               Server
|—— FIN (Seq=u) —————> |
|<— ACK (Ack=u+1) ————— |
|<— FIN (Seq=v) ————— |
|—— ACK (Ack=v+1) —————> |
```

Q9. TCP Congestion Control: AIMD with given scenario

Initial ssthreshold = 32

Stages:

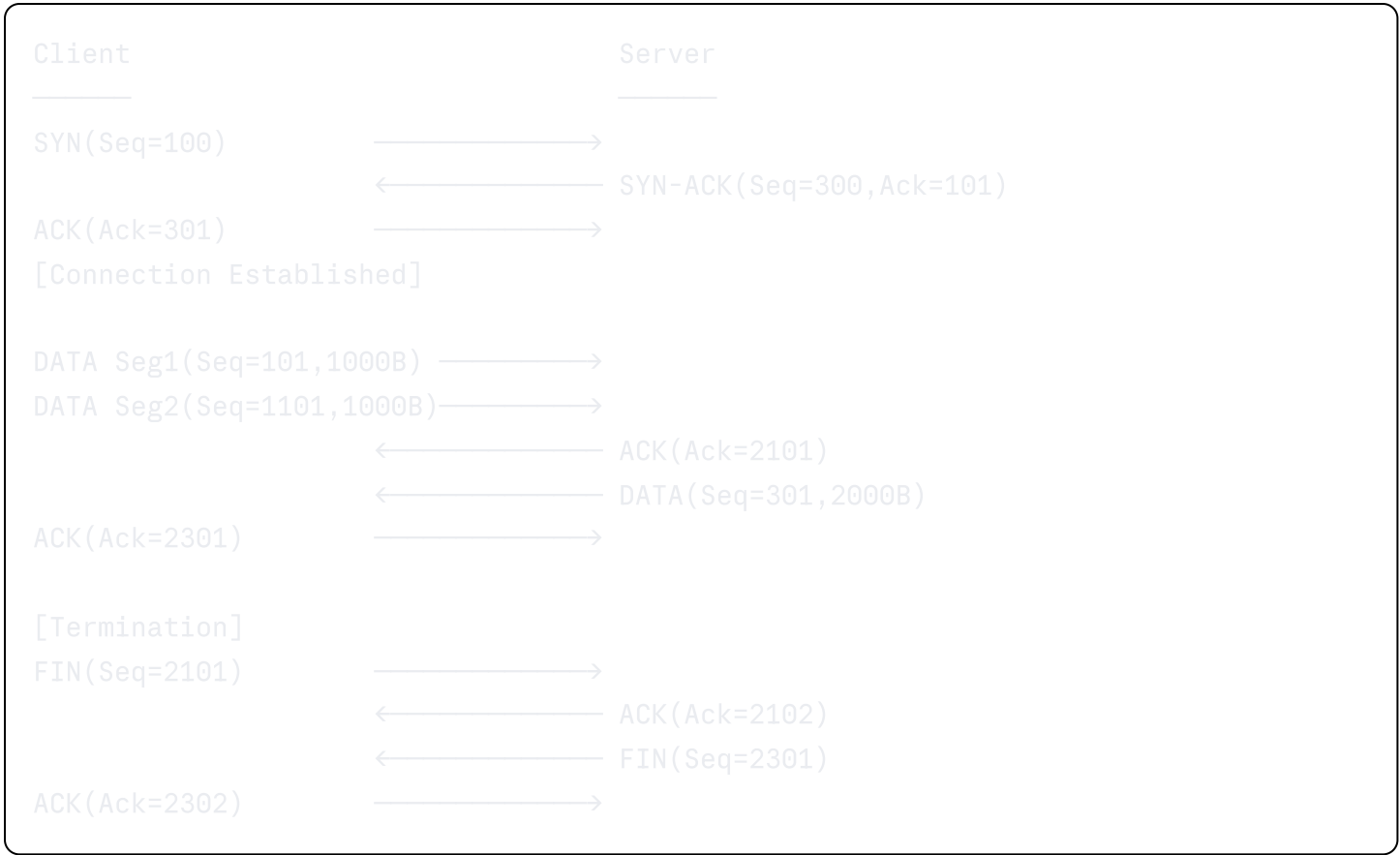
1. **SS (Slow Start):** cwnd doubles each round (1,2,4,8,...) until ssthreshold or timeout
2. **CA (Congestion Avoidance):** cwnd += 1 per RTT (additive increase)
3. On **Timeout:** ssthreshold = cwnd/2, cwnd = 1, restart SS
4. On **3 Duplicate ACKs:** ssthreshold = cwnd/2, cwnd = ssthreshold (fast recovery)

Given Scenario:

Round	Stage	cwnd	Event
1	SS	1	
2	SS	2	
3	SS	4	
4	SS	8	
5	SS	16	
6	SS	32	
7	CA	33	
...	CA	...	
cwnd=40	—	40	TIMEOUT → ssthresh=20, cwnd=1
Restart SS	1,2,4,...20	→ CA	
CA	21,22,23,24	cwnd=24	TIMEOUT → ssthresh=12, cwnd=1
Restart SS	1,2,4,8,12	→ CA	
CA	13,...,36	cwnd=36	3-ACK → ssthresh=18, cwnd=18

Q10. TCP Full Exchange: Client sends 2000 bytes in 2 segments, Server sends 2000 bytes in 1 segment

Assume Client ISN=100, Server ISN=300



Q11. TCP AIMD: Find cwnd at end of 10th transmission

Initial window = 2 MSS, ssthreshold = 8 MSS

Transmission	cwnd (MSS)	Stage
1	2	SS
2	4	SS
3	8	SS → ssthreshold reached
4	9	CA (+1)
5	10	CA → TIMEOUT
—	ssthresh = 5, cwnd=1	Reset
6	1	SS
7	2	SS
8	4	SS
9	5	SS → ssthresh
10	6	CA

cwnd at end of 10th transmission = 6 MSS

Q12. UDP Checksum Calculation

(Based on the image values in the original document — apply the pseudo header method: sum all 16-bit words of the pseudo header + UDP header + data, add carry bits, take 1's complement.)

SECTION E: Application Layer & QoS

Q1. What is DNS?

DNS (Domain Name System) translates human-readable domain names (e.g., www.google.com) to IP addresses (e.g., 142.250.190.36).

Purpose:

- Humans remember names; computers use IP addresses
 - DNS provides a distributed, hierarchical database for this translation
 - Eliminates need for a single central host file
-

Q2. DNS Record Types

Record	Full Name	Purpose
A	Address	Maps domain to IPv4 address
AAAA	—	Maps domain to IPv6 address
MX	Mail Exchange	Specifies mail server for domain
CNAME	Canonical Name	Alias for another domain name
NS	Name Server	Specifies authoritative DNS server
PTR	Pointer	Reverse DNS (IP → name)
SOA	Start of Authority	Zone info, primary NS, serial
TXT	Text	Human-readable / SPF / DKIM records

Q3. Recursive vs. Iterative DNS Query

Feature	Recursive Query	Iterative Query
Who resolves	DNS resolver does all work	Client queries each server in sequence
Server behavior	Server queries other servers on behalf of client	Server returns referral to next server
Used between	Client ↔ Local DNS Resolver	Local Resolver ↔ Root/TLD/Authoritative
Load	More load on resolver	More load on client

Hybrid (typical): Client sends recursive to local resolver; resolver uses iterative with root and other servers.

Q4. Email Working

Components:

- **UA (User Agent):** Email client (Outlook, Gmail)
- **MTA (Mail Transfer Agent):** Routes email (Sendmail, Postfix)
- **MAA (Mail Access Agent):** Retrieves email (POP3/IMAP)

Process:

Sender's UA —SMTP→ Sender's MTA —SMTP→ Receiver's MTA —POP3/IMAP→ Receiver's UA

1. User composes email in UA
 2. UA submits to MTA via SMTP (port 25)
 3. MTA routes to recipient's MTA using DNS MX records
 4. Recipient's MTA stores email
 5. Recipient's UA retrieves via POP3 (port 110) or IMAP (port 143)
-

Q5. Role of UA and MTA in SMTP

UA (User Agent):

- Interface for composing, reading, sending emails
- Submits email to MTA using SMTP
- Example: Microsoft Outlook, Thunderbird, Gmail web UI

MTA (Mail Transfer Agent):

- Stores and forwards emails using SMTP
- Queries DNS for MX records to find destination MTA
- Example: Sendmail, Postfix, Microsoft Exchange

Example:

- User A (UA=Outlook) composes email to B@example.com
 - Outlook (UA) submits to smtp.a.com (MTA-A) via SMTP
 - MTA-A queries DNS: MX for example.com → mail.example.com
 - MTA-A connects to mail.example.com (MTA-B) via SMTP
 - MTA-B stores mail; User B's UA (IMAP) retrieves it
-

Q6. Convert 01001100 01101001 01111110 to Base64

Step 1: Group 24 bits (3 bytes) = 01001100 01101001 01111110

Step 2: Split into four 6-bit groups: 010011 | 000110 | 100101 | 111110

Step 3: Convert each to decimal:

- 010011 = 19 → T
- 000110 = 6 → G
- 100101 = 37 → l
- 111110 = 62 → +

(Using Base64 table: A=0,B=1,...,Z=25, a=26,...,z=51, 0=52,...,9=61, +=62, /=63)

- 19 → T
- 6 → G
- 37 → l
- 62 → +

Base64 result = TGl+

(Check: 01001100=L in ASCII, 01101001=i, 01111110=~; "Li~" encodes to "TGl+")

Q7. Quoted-Printable Encoding

Used in email to encode non-ASCII characters in a printable ASCII format.

Rules:

- ASCII printable characters (33-126) pass through unchanged
- Non-printable/non-ASCII byte → encoded as =XX (hex)
- Lines must not exceed 76 characters; soft line break = = at end

Example:

- Plain text: "Héllo" (é = 0xE9)
- Encoded: **H=E9llo**

Another example: Tab (0x09) → =09, Newline uses CRLF

Q8. HTTP GET and PUT Methods

GET

- Retrieves data from server
- Parameters in URL query string
- Idempotent; no body

```
GET /index.html HTTP/1.1
Host: www.example.com
```

Server responds with 200 OK + HTML content

PUT

- Uploads/replaces resource at specified URL
- Request body contains data to store
- Idempotent

```
PUT /files/photo.jpg HTTP/1.1
Host: www.example.com
Content-Length: 12345
[binary data]
```

Server responds with 200 OK or 201 Created

Q9. HTTP Response Time: 8 embedded objects, RTT=20ms

Assumptions:

- 1 RTT to establish TCP connection
- 1 RTT to send request and receive first byte of response
- Transmission time ≈ 0 (simplification)

Non-Persistent HTTP

- Each object requires: 2 RTT (1 for TCP setup + 1 for request/response)
- Base HTML page: 2 RTT
- 8 embedded objects (sequential): $8 \times 2 \text{ RTT} = 16 \text{ RTT}$
- **Total = 2 + 16 = 18 RTT = $18 \times 20\text{ms} = 360\text{ms}$**

Persistent HTTP (with pipelining)

- 1 RTT for TCP setup
- 1 RTT for base HTML
- 1 RTT for all 8 embedded objects (pipelined)
- **Total = 3 RTT = $3 \times 20\text{ms} = 60\text{ms}$**

Comment: Persistent HTTP is $6\times$ faster than non-persistent for this scenario.

Q10. Priority Queue vs. Weighted Fair Queue

Priority Queue

- Multiple queues with priority levels (High, Medium, Low)
- Higher priority queue always served first
- Low-priority traffic can starve if high-priority is always present
- Used for voice/video (high) vs. best-effort (low)

Weighted Fair Queue (WFQ)

- Each flow/class gets a weight
- Bandwidth allocated proportionally to weights
- Prevents starvation — every queue gets some service
- Example: Voice=50%, Video=30%, Data=20%

Q11. QoS and Traffic Shaping - Leaky Bucket

QoS (Quality of Service) ensures network resources are allocated to meet performance requirements.

QoS Factors:

1. **Bandwidth** - minimum guaranteed data rate
2. **Delay** - max acceptable end-to-end delay
3. **Jitter** - variation in delay (critical for voice/video)
4. **Packet Loss** - acceptable loss rate

Leaky Bucket Algorithm

Regulates bursty traffic into a smooth, constant-rate output.

```
Incoming packets (bursty)
    ↓
  [Bucket/Queue] (capacity B)
    ↓ (constant rate r)
Output stream (smooth)
```

- Packets enter at variable rate; exit at constant rate r
- If bucket overflows $\rightarrow$ packet dropped
- Ensures no burst exceeds bucket capacity
- **Advantage:** Smooth output; **Disadvantage:** Cannot accommodate short bursts

Q12. Weighted Fair Queue (WFQ) with Diagram

WFQ extends fair queuing by assigning weights to flows:

```
Flow 1 (weight=5) —→ [Q1] —┐
Flow 2 (weight=3) —→ [Q2] —┴— [Scheduler] —→ Output link
Flow 3 (weight=2) —→ [Q3] —┐
```

Bandwidth allocation:

- Total weight = $5+3+2 = 10$
- Flow 1 gets 50%, Flow 2 gets 30%, Flow 3 gets 20% of bandwidth

Scheduler serves packets in proportion to weights, ensuring fairness while differentiating services.

Q13. IntServ vs. DiffServ

Feature	IntServ	DiffServ
Approach	Per-flow reservation	Class-based
Protocol	RSVP (Resource Reservation)	DSCP marking in IP header
Scalability	Poor (state per flow)	High (aggregated classes)
Guarantee	Hard QoS (guaranteed)	Soft QoS (best-effort class)
Deployment	Complex	Simple; widely deployed
Granularity	Individual flow	Traffic class/aggregate
Example use	Video conferencing	Enterprise WAN prioritization

Q14. Token Bucket Algorithm

More flexible than Leaky Bucket; allows controlled bursts.

Mechanism:

- Tokens generated at rate r tokens/second
- Bucket capacity = B tokens (max burst)
- Each packet consumes tokens proportional to size
- If enough tokens $\rightarrow$ send immediately; else wait or discard



Key difference from Leaky Bucket:

- Leaky Bucket: output always at constant rate
- Token Bucket: allows burst up to B tokens, average rate = r

Applications: Traffic policing, traffic shaping in ISPs, QoS enforcement.

End of Answer Bank - CS30003 Computer Networks